[Experiment, Analysis, and Benchmark] Right by Accident: Text-to-SQL Gold Queries Whose Answers Depend on the Database State

Jiacheng Ding[1], Cong Guo[1], Peihua Mai[2,*]
[1]University of Memphis, Memphis, TN, USA
[2]National University of Singapore, Singapore
jding2@memphis.edu, cguo@memphis.edu, peihua.m@u.nus.edu

Abstract—Text-to-SQL benchmarks score a system by running its query and a reference query, the gold query, on one database and comparing the results. Each gold query is therefore checked against a single database state. We study gold queries whose answer depends on that state in a way the question does not intend. Some cut a ranked list at a tie and return whichever tied row the engine produces first. Some count join rows where the question asks for entities. Some return nothing, so the benchmark accepts any prediction that also matches nothing. We define such state-dependent gold queries and present GoldCheck, a deterministic checker that reports a finding only with a proof by execution. The proof is a second reading of the query that disagrees with the gold on the official database, or a one-cell edit of the database on which the two readings disagree. In three independent human reviews of the BIRD gold, the review changed the output of flagged queries 2.3 to 4.1 times as often as the output of other queries. On 1,036 examples that we never inspected, the ratio is 2.4, with a 95% interval from 1.7 to 3.3. Static SQL rules and one-time execution reach at most 1.7. The latest official BIRD Dev release still contains 65 flagged queries. For 31 of them, the test is defective whatever the question intends, and two annotators independently judged 26 of the other 34 to be errors. GoldCheck needs a median of 17 ms per query and no language model. Rebuilding the tables without changing the data changes the gold answer of 12 of the 21 tie queries. An LLM reviewer names more count fan-outs as it grows from 3B to 120B parameters, but it names at most 11 of 41 ties unless it receives the evidence from GoldCheck.

Index Terms—text-to-SQL, benchmarks, data quality, query testing, test data generation

I. Introduction

Large language models (LLMs) have renewed interest in how people and programs talk to databases. Text-to-SQL translates a question into an executable query. It is the most direct form of this interaction, and both the database and the language communities now build and compare such systems [1]–[3]. Researchers measure progress on benchmarks such as BIRD [4] and Spider [5]. Each example pairs a question with a database and a reference query, the gold query. A benchmark scores a system's query as correct when it returns the same result as the gold query on the benchmark database. Leaderboard numbers therefore rest on the gold queries.

People write gold queries, and people make errors. A recent audit found errors in more than half of the examples of BIRD Mini-Dev and Spider 2.0-Snow and showed that corrections reorder a leaderboard [6]. The BIRD maintainers released a reviewed Dev set in November 2025 [7]. These reviews are expensive. The audit used an LLM agent, SAR-Agent, that costs $0.44 per example with OpenAI o3, and two authors checked every report it produced [6]. Five researchers led the maintainers' review with a larger group of reviewers [7].

This paper studies one kind of gold error that is easy to make, hard to see, and cheap to find automatically. An annotator writes a gold query and checks it on one database. If it returns a plausible answer there, the annotator accepts it. But a query can return the expected answer on that one database by accident. Fig. 1 shows three gold queries from the latest official BIRD Dev release.

- "Who is the tallest superhero?" The gold query sorts by height and keeps one row. Two superheroes, Surtur and Ymir, share the largest height of 30,480 cm. The gold answer is Surtur because SQLite happens to return that row first. A system that lists both receives no credit.
- "How many female patients have a white blood cell count below 3.5?" The gold query joins patients to lab tests and counts patient identifiers without DISTINCT. It returns 641, the number of lab tests. The number of patients is 88.
- "What is the average total price of the transactions in January 2012?" The database has no such transactions. The gold answer is NULL, the value an average takes over no rows. A system whose query also matches no rows receives full credit, for example a query that filters on a wrong date format.

In each case, running the gold query raises no flag. It returns one plausible value. The problem lies in how the answer depends on the data.

*Corresponding author.

(a) Tie at the cut. Who is the tallest superhero?

```
SELECT superhero_name FROM superhero
ORDER BY height_cm DESC LIMIT 1 -- Surtur
-- Ymir has the same height and is cut off
```

(b) Count fan-out. How many female patients have a white blood cell count below 3.5?

```
SELECT COUNT(T1.ID)
FROM Patient T1 JOIN Laboratory T2 ON T1.ID = T2.ID
WHERE T2.WBC < 3.5 AND T1.SEX = 'F'
-- 641; COUNT(DISTINCT T1.ID) is 88
```

(c) Empty answer. What is the average total price of the transactions in January 2012?

```
SELECT AVG(Price) FROM transactions_1k
WHERE Date LIKE '2012-01%' -- NULL
```

Fig. 1. Three gold queries from the 2025-11-06 BIRD Dev release, lightly abridged. Comments show what GoldCheck reports.

We call such queries state-dependent. The database community has long known that one database state is a weak test of a query. Test data generation for SQL grading [8], test suites for text-to-SQL predictions [9], and database-engine testing [10], [11] all vary the state. The gold query is the oracle for everything else, yet benchmarks still check it on one state. Jin et al. noticed this in passing and hand-inserted distinguishing rows for 6 of the 100 Dev examples they corrected [6]. We turn that observation into a systematic check.

Approach. GoldCheck targets three SQL constructs whose meaning depends on the data. They are a ranked list cut by LIMIT, a count over a join, and an empty answer. For each construct, GoldCheck builds a second reading of the gold query and runs both readings on the official database. It reports a finding only when it can show a concrete difference. When the official data happen not to separate the readings, it edits one cell to produce a state that does. Every finding comes with an executable witness. A person still decides whether the finding is an error, because only the question says which reading it intends.

GoldCheck is small. It uses no language model, gives the same answer on every run, and checks the 1,534 BIRD Dev queries in about a minute. Our goal is to measure how far a benchmark can trust a single database state as a test, and to provide a check that makes the gap visible before a person or an LLM agent spends effort on a review.

Contributions. This paper makes four contributions.

(1) Problem. We define state-dependent gold queries. A live finding is visible on the official database, and a latent finding needs a one-cell change.
(2) Checker. GoldCheck detects four classes of live findings and synthesizes one-cell witnesses for latent ones. It proves each finding by execution, with a median cost of 17 ms per query.
(3) Comparison with human reviews and baselines. In three independent reviews of the BIRD gold, the review changed the output of 35% to 57% of flagged queries, 2.3 to 4.1 times the rate of other queries. Static rules and one-time execution reach at most 1.7.
(4) Evidence that the cause is the test design. Rebuilding the tables of the latest release without changing the data changes 12 of its 21 tie answers. SAR-Agent on open models from 3B to 120B parameters names at most 11 of 41 ties, and 27 to 37 once it receives the evidence from GoldCheck.

We also report three case studies, on predictions of four open models, on the BIRD training split, and on a prior ranking analysis. We release GoldCheck, the aligned corpus, and every script behind each number at https://github.com/graphuofm/goldcheck.

II. Preliminaries and Related Work

A. Execution-based scoring

BIRD [4] has questions over 95 databases. Its Dev split has 1,534 questions over 11 SQLite [12] databases from domains such as schools, banking, medicine, motorsport, and card games. Each example has a question, an optional evidence note with a domain convention, a gold query, and a difficulty label. BIRD scores a prediction P by execution match on the official database D. It counts P as correct if and only if the result sets of $P(D)$ and $G(D)$ are equal, where G is the gold query. Mini-Dev is a 500-example subset of Dev.

B. State-dependent gold queries

Let q be a question, D the official database, and G the gold query. Execution match compares a prediction with $G(D)$ and nothing else. The test is only as good as the pair (G, D). If what q asks does not determine $G(D)$, or if $G(D)$ cannot separate a right prediction from a wrong one, the test fails silently.

Some SQL constructs admit more than one reasonable reading. One row or every row with the largest height can answer "the tallest superhero". A count of rows or a count of distinct patients can answer "how many patients". We write G_c for the gold query with construct c replaced by its other reading.

Definition 1 (Live finding): G has a live finding on D if (i) $G(D) \neq G_c(D)$ for some construct c, (ii) G cuts a ranked list and the first ranked row has a NULL sort key, or (iii) $G(D)$ is empty or contains only NULL values.

Example. For the tallest superhero in Fig. 1(a), $G(D)$ is $\{\text{Surtur}\}$ and the tie-complete reading $G_c(D)$ is $\{\text{Surtur}, \text{Ymir}\}$. This is a live finding of case (i). The query in Fig. 1(c) returns only NULL and is a live finding of case (iii).

Definition 2 (Latent finding): G has a latent finding on D if $G(D) = G_c(D)$ but there is a database D' that differs from D in a single cell such that $G(D') \neq G_c(D')$.

Example. A gold query returns the phone number of the school with the most test takers. On D, one school

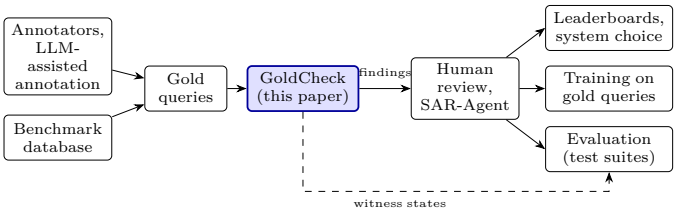


Fig. 2. Life cycle of a gold query and the position of GoldCheck. The dashed arrow marks witness states that can extend the evaluation.

has the largest count, so $G(D) = G_c(D)$. If the count of a second school is set to the same value, G returns one phone number and G_c returns two.

Terms. We use four terms with one meaning each. A finding is a live or latent finding as defined above. A flagged query is a gold query with at least one live finding. A defective test is a flagged query whose test fails whatever the question intends, which holds for ties and empty answers. An error is a flagged query that a person judged wrong from the question. A finding proves that two readings differ on a concrete database. It does not prove an error, because only the question says which reading it intends.

Role of the engine. The live cases fail in different ways. (1) Ties. The SQL standard leaves the order of rows with equal sort keys to the implementation. A query with ORDER BY o LIMIT k does not define its result when the k-th and $(k{+}1)$-th rows share the value of o. The engine returns whichever tied row it meets first. (2) NULL at the top. Engines disagree on where NULL sorts. SQLite and MySQL place it before every other value in ascending order, and PostgreSQL places it after. (3) Fan-out and empty answers. These are deterministic. A count over a join returns the number of join rows. It equals the number of entities only when no entity matches more than one row. An empty answer makes the test vacuous, since the gold agrees with any query that also matches nothing.

Scope. We chose the classes from the research audit of Mini-Dev [6]. In our coding of its 134 corrections, ties at a top-row selection (10 corrections) and missing DISTINCT after a join (9) are among the five most frequent of 42 mechanisms. We kept only constructs whose second reading follows mechanically from the parse tree and that execution can compare. Mechanisms that need knowledge of the domain or of the question, such as computing an age by subtracting years, are out of scope.

C. Related work

Fig. 2 places GoldCheck in the life cycle of a gold query. Upstream, people write gold queries against a database, increasingly with LLM help. In the middle, reviewers and audit tools check them. Downstream, leaderboards, evaluation methods, and model training consume them.

Upstream. In Spider [5] and BIRD [4], annotators wrote a query and checked it on the benchmark database. Spider 2.0 moved to enterprise workflows [3]. Jin et al. propose SAPAR, an annotation pipeline in which an LLM agent reviews each annotation and people confirm or reject its diagnosis [6]. In all of these, the checks run on the database state that ships with the benchmark.

Midstream. Wretblad et al. found noisy questions and gold queries in BIRD's financial database [13]. Arcwise practitioners re-checked Mini-Dev by hand [14]. Jin et al. audited Mini-Dev and Spider 2.0-Snow with SAR-Agent [6]. In each step, SAR-Agent proposes a verification query, runs it, and reads the result. At the end it writes a report on correctness and ambiguity. With o3, SAR-Agent reached 83% precision on Mini-Dev at $0.44 per example. GBV-SQL reports a typology of gold errors that come from the SQL, the question, or the database [15]. In machine learning, label errors in test sets are a known problem [16]. These methods are broad. They can find a misread question or a wrong table. Their judgments come from people or from an LLM, so they are costly, can vary from run to run, and need verification. GoldCheck is narrow and runs before them. It covers only state-dependent errors, but each finding is reproducible and carries its own proof.

Downstream. Leaderboards rank systems by execution accuracy, and experimental studies in the database community compare systems on them [1], [17]–[19]. Zhong et al. showed that one database lets wrong predictions pass and built distilled test suites of databases [9]. Pourreza and Rafiei analyzed acceptable predictions that the benchmark judged wrong [20], and FLEX uses an LLM to emulate expert judgment of predictions [21]. These works assume that the gold query is right and improve how a scorer compares predictions with it. Gold queries also serve as training data for fine-tuned systems such as CodeS [19]. A state-dependent gold query misleads all of them. GoldCheck's latent witnesses are test-suite states aimed at the gold, and a benchmark can add them to its evaluation.

Principles we borrow. SQL equivalence is undecidable in general. Cosette [22] and EQUITAS [23] prove equivalence for restricted fragments. XData generates databases that separate a query from its likely mutants [8], and engine-testing tools vary queries and states to find engine bugs [10], [11]. Data-cleaning research has shown that no single detector finds most errors and that a review budget is best spent on a combination of detectors [24]–[27]. GoldCheck applies these ideas to a new target. It holds the engine fixed and asks whether the benchmark data determine the result of the reference query. Table I summarizes the differences.

III. Solution Overview

GoldCheck takes a gold query G and the path of its database D, and returns a list of findings with evidence. Fig. 3 shows its five components. (1) The parser turns G into a parse tree and checks that printing the tree back gives the same result as G. (2) The reading builder rewrites

TABLE I
Approaches to checking gold queries. The cost of SAR-Agent with o3 is the figure reported by its authors [6].

	Scope	LLM	Determ.	Evidence	Cost/query
Human review	broad	no	no	note	manual
SAR-Agent (o3)	broad	yes	no	report	$0.44
SAR-Agent (open)	broad	yes	no	report	6–82 s GPU
Static rules	narrow	no	yes	none	<1 ms
Run once	narrow	no	yes	result	ms
Random states	narrow	no	seeded	witness	varies
GoldCheck	narrow	no	yes	witness	17 ms

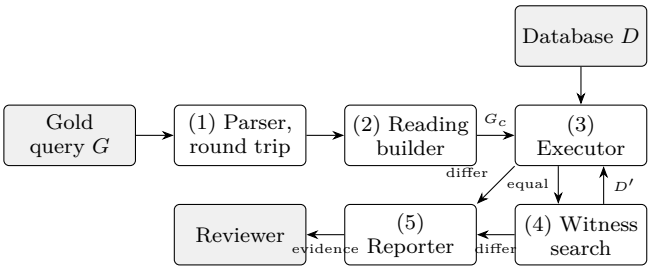


Fig. 3. Components of GoldCheck. Grey boxes are the inputs and the consumer. G_c is the other reading of G, and D' is D with one cell edited. "Equal" and "differ" refer to the results of G and G_c.

the tree into the other reading G_c of each supported construct. It builds a tie-complete query for a top-k cut and a COUNT(DISTINCT) query for a count of a primary key over a join. (3) The executor runs G and each G_c on D through a read-only connection and compares the result sets. A difference is a live finding. (4) The witness search runs only when a top-k query shows no difference on D. It edits one cell in a temporary copy of one table and asks the executor to run G and G_c again. A difference is a latent finding. (5) The reporter attaches to each finding the SQL of both readings and their results, so that a reviewer can re-run the proof.

The components share one rule. GoldCheck reports a finding only when the executor has observed a difference on a concrete database state. A person then decides whether the finding is an error.

IV. The GoldCheck Checker

GoldCheck takes a gold query and the path of its SQLite database. It parses the query with sqlglot [28], runs it, and applies the checks below. It reports a finding only with evidence obtained by running SQL. GoldCheck makes every edit in TEMP shadow tables on a read-only connection and rolls it back at the end of a savepoint.

Round trip. Each check rewrites the parse tree and prints it as SQL. A difference must come from the rewrite and not from the parser. GoldCheck therefore first prints the unmodified tree and runs it. If its result differs from the original, GoldCheck skips the structural checks. This happens for 3.3% of the 2025 Dev gold queries.

Top-k cut. If the outermost SELECT has ORDER BY o LIMIT k, GoldCheck builds the tie-complete reading

school	test takers	phone
s_1	5,837	p_1
s_2	5,837 (was 2,125)	p_2
…	…	…

cut at LIMIT 1

Official D: G and G_c both return $\{p_1\}$.
Edited D': G returns $\{p_1\}$, G_c returns $\{p_1, p_2\}$.

Fig. 4. A latent witness for the phone number of the school with the most test takers over 1500, with schools and phone numbers abbreviated.

```
SELECT c1..cm FROM (SELECT <outputs> AS c1..cm,
  RANK() OVER (ORDER BY o) AS r FROM ... ) WHERE r
      <= k
```

which keeps every row tied with the k-th row, with the same filters, joins, and grouping. GoldCheck replaces sort keys that refer to output aliases by their expressions. If $G_c(D) \supsetneq G(D)$, the gold drops tied rows whose output differs, and GoldCheck reports a top-k tie with the dropped rows. It also re-runs the query without the limit and with the sort keys added to the output. If the first key is NULL, it reports NULL at top. GoldCheck does not rewrite queries with DISTINCT or SELECT * at the top level.

Count fan-out. For every COUNT($t.a$) without DISTINCT in a SELECT with joins, GoldCheck checks whether a is the single-column primary key of t. If so, the author meant to count entities of t, and a join can repeat them. GoldCheck runs the query with COUNT(DISTINCT $t.a$). If the result changes, it reports a count fan-out with both values.

Empty answer. GoldCheck reports an empty answer if the gold result has no rows or only NULL values.

Targeted latent witnesses. If the tie-complete reading returns the same result as the gold, the official data have no harmful tie at the cut. GoldCheck then builds one with a single-cell edit. It requires every sort key to be a plain column of one base table T. It runs the query without the limit and adds the sort keys and the rowid of T to the output. The first rows below the cut are exactly the rows that a tie could promote. For each of up to six such rows, GoldCheck sets the sort key in a shadow copy of T to the value of the k-th row and runs G and G_c. It stops at the first state on which they differ. The witness records the table, the rowid, the column, the old and new value, and both results. Consider "What is the phone number of the school with the highest number of test takers scoring over 1500?" Setting one school's count from 2,125 to 5,837 makes the gold return one phone number and the tie-complete reading two.

Algorithm 1 summarizes the procedure, and Fig. 4 shows a latent witness.

Design choices. Three choices keep findings few and meaningful. First, a tie counts only if the tied rows differ in the output columns. Ties at the cut are common, and

Algorithm 1 GoldCheck for one gold query

Require: gold query G, official database D (opened read-only)
$F \leftarrow \emptyset$; $T \leftarrow \text{Parse}(G)$; $R \leftarrow G(D)$
if R is empty or contains only NULL then
  $F \leftarrow F \cup \{\text{empty answer}\}$
end if
if $\text{Print}(T)(D) \neq R$ then return F ▷ round trip fails
end if
if T has an outer ORDER BY o LIMIT k then
  $G_c \leftarrow \text{TieComplete}(T)$
  if $G_c(D) \supsetneq R$ then
    $F \leftarrow F \cup \{\text{tie} : G_c(D) \setminus R\}$
  end if
  if the first sort key without the limit is NULL then
    $F \leftarrow F \cup \{\text{NULL at top}\}$
  end if
  if $G_c(D) = R$ and o are plain columns of a base table B then
    copy B into a TEMP shadow table, keeping rowid
    for each of the first six rows r below the cut do
      savepoint; set $r.o$ to the sort key of the k-th row
      if $G(D') \neq G_c(D')$ then
        $F \leftarrow F \cup \{\text{latent} : (B, r, o)\}$; break
      end if
      rollback
    end for
  end if
end if
for each COUNT($t.a$) without DISTINCT under a join do
  if a is the one-column primary key of t then
    if G with COUNT(DISTINCT $t.a$) returns $\neq R$ then
      $F \leftarrow F \cup \{\text{fan-out}\}$
    end if
  end if
end for
return F, each finding with the SQL and results that prove it

many are harmless. Two rows of the same entity can share a sort key and print the same answer. Second, a fan-out counts only when the counted column is the primary key of its table. Counting a primary key signals that the author meant to count entities, so a join that repeats them changes the meaning. Counting a non-key column over a join is often deliberate. Third, GoldCheck edits only rows just below the cut, and only their sort key. Only such edits can change which rows survive the cut, so only they can create a tie there. Our experiments measure the first and third choices.

Complexity. Let T_G be the time to run G on D. GoldCheck runs G once, its printed tree once, the tie-complete reading once, the unlimited probe once, and one DISTINCT reading per counted primary key. The witness search adds at most six edits with two runs each and one copy of the base table B. The total is at most $(16+c)$ query executions, where c is the number of counted primary keys, plus $O(|B|)$ for the copy. The cost is therefore linear in the cost of the gold query itself and does not depend on the number of gold queries checked before.

Guarantees and limits. GoldCheck observes every reported difference on a concrete database state, so its findings are sound by construction. GoldCheck does not judge which reading the question intends and does not prove equivalence. It misses differences that its rewrites and edits do not reach. It does not look for errors that are unrelated to the data, such as a wrong table or a misread question. GoldCheck is about 240 lines of Python on top of sqlglot and the sqlite3 module. It registers an IIF function, which our SQLite version 3.31 lacks.

TABLE II
Aligned releases. "Changed" counts gold queries whose normalized text differs. "Output" counts those whose two versions return different result sets.

Review, old to new	n	Changed	Output
Mini-Dev T0 to T1, community	498	97	89
Mini-Dev T1 to T2, research audit	498	134	75
Dev 2024 to 2025, maintainers	1,534	396	252
held-out, not in Mini-Dev	1,036	273	176
Dev-100 old to corrected, audit	100	69	19

V. Experimental Evaluation

A. Releases

Table II lists the releases we align. For Mini-Dev, three versions of the same 498 SQLite examples exist. T0 is the gold shipped by BIRD. T1 is a community correction by practitioners at Arcwise [14]. T2 is the research audit of Jin et al. [6]. The audit started from T1 and released a per-example diff. We verified that its recorded starting point equals T1 for all 498 examples. For the full Dev set, two versions exist. They are the 2024-06-27 gold and the maintainers' review of 2025-11-06 [7]. BIRD drew Mini-Dev from Dev. We call the other 1,036 Dev examples held-out, and we built no definition by looking at them. A 2026 Mini-Dev release [29] rewrote and renumbered the questions, so we cannot align it. We also use Dev-100. It consists of 100 Dev questions that Jin et al. corrected, with predictions from 16 published systems [6].

A changed gold query is not always a wrong one. Reviewers also rename output columns and rewrite for style. We therefore use the stricter outcome, a change of output, wherever it matters. To compare text, we parse both versions with sqlglot in the SQLite dialect and print them in canonical form.

B. Methods and measures

Releases and references. We run every method on the version under review in each of three human reviews (Table II). We then count how often the review changed the output of the queries the method flags. None of the reviews used any of the methods. The maintainers' Dev review is the cleanest reference. Mechanisms we saw in the Mini-Dev audit motivated our classes, and Dev contains 1,036 examples outside Mini-Dev that we did not inspect when we chose the classes. We report results on these held-out examples separately as an out-of-sample test.

Methods. We chose baselines by one criterion. Each removes one ingredient of GoldCheck or represents a published way to check or test queries. The first three are our own constructions and do not stand for a published system. (1) Static rules read the SQL without running

it. They flag the same constructs as GoldCheck, a top-level ORDER BY … LIMIT and a COUNT of a column without DISTINCT over a join. (2) Run once mirrors what an annotator sees. It flags a gold query that fails, returns nothing, returns duplicate rows, or returns a NULL. (3) Random edits replace GoldCheck's targeted witness search with random edits in the style of test suites. Each try sets the sort key of a random row to the boundary value. (4) SAR-Agent [6] runs with its released prompt and loop on open models. This re-run is not the published system, which uses o3, and we label it as an open-model re-run throughout.

Measures. For a set of flagged queries, we report the share whose output the next review changed. We divide this share by the share among all other queries and call the ratio lift. We also report the recall of all output changes, and we report cost as time per query. Lift is a ratio, so we checked its denominator. The share among unflagged queries lies between 14% and 16% in the three reviews and is never close to zero. We report the absolute shares next to each lift and give 95% intervals from 2,000 bootstrap resamples of the queries [30]. Figures show Wilson intervals [31].

C. Agreement with human reviews

Table III gives the results. GoldCheck flags few queries, and the reviews changed many of them. They changed the output of 56%, 57%, and 35% of flagged queries, against 14% to 15% of the rest. The lift is 3.7, 4.1, and 2.3. Static rules flag 4.7 to 11 times as many queries, with a lift of 1.4, 1.0, and 1.6. A top-k cut or a count over a join is common and usually harmless. What matters is whether the data make the reading matter, and only execution shows that. Running the gold once gives a lift of 1.0 to 1.7. Fig. 5 shows the GoldCheck rates with intervals.

Uncertainty and out-of-sample test. The 95% bootstrap intervals of the GoldCheck lift are 2.5 to 5.3, 2.1 to 6.5, and 1.7 to 3.0 for the three reviews. The intervals of run once are 0.5 to 2.0, 0.8 to 2.8, and 0.7 to 1.5, and those of static rules are 1.0 to 2.1, 0.6 to 1.5, and 1.2 to 2.0. On the 1,036 held-out Dev examples, GoldCheck flags 72 queries. The review changed the output of 27 of them (37.5%) and of 149 of the other 964 (15.5%). The lift is 2.4, with an interval from 1.7 to 3.3. Static rules reach 1.7 there (1.3 to 2.3) and run once 1.0 (0.6 to 1.5). The intervals of GoldCheck and static rules overlap on the maintainers' review, so on that review we can only state that GoldCheck flags one fifth as many queries at a higher changed share (35.2% against 21.9%).

Latent witnesses behave differently. The reviews changed queries whose only finding is latent at or below the base rate, at 6%, 12%, and 13%. Reviewers do not treat a top-k query as wrong when the data happen to have no tie. This matches the definition. A latent finding shows that the test is weak, not that the gold is wrong.

TABLE III
Gold queries flagged per method. Out% is the share whose output the next review changed, lift is its ratio to unflagged queries, and recall covers all output changes. Bold marks the highest value per review.

Review	Method	Flagged	Out%	Lift	Recall
Community (Mini-Dev T0)	Static rules	175	22.3	1.44	39/89
	Run once	40	20.0	1.13	8/89
	GoldCheck	32	56.2	3.69	18/89
Research audit (Mini-Dev T1)	Static rules	154	14.9	0.99	23/75
	Run once	38	23.7	1.65	9/75
	GoldCheck	14	57.1	4.13	8/75
Maintainers (Dev 2024)	Static rules	498	21.9	1.59	109/252
	Run once	157	17.2	1.05	27/252
	GoldCheck	105	35.2	2.34	37/252

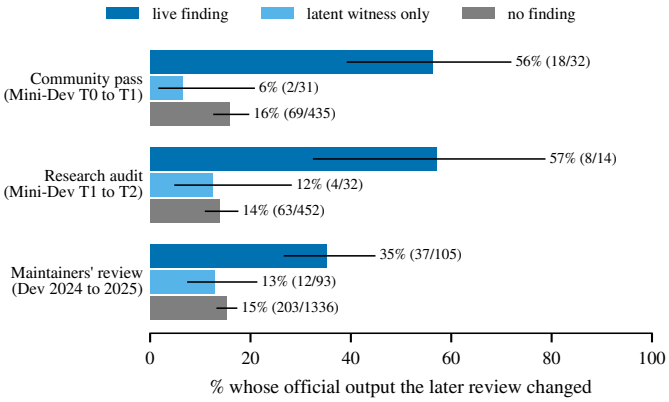


Fig. 5. Share of gold queries whose output the next human review changed, by GoldCheck result on the version under review, with 95% Wilson intervals.

D. Survival of findings across reviews

Fig. 6 follows each live finding into the next version, and Table IV counts findings per release. The two Mini-Dev reviews together removed 29 of 34 findings, including 12 of 13 ties and 14 of 17 count fan-outs. The maintainers' Dev review removed 47 of 110 and none of the eight empty answers. Of the 65 Dev queries with a live finding in the 2025 release, GoldCheck had already flagged 60 in 2024. The review left 55 of them word for word unchanged. Five findings remain in Mini-Dev after the audit that SAR-Agent assisted. Two are clear errors. In one, the gold picks the first race of the youngest driver from tied rows rather than by date. The other three are counts meant to count join rows.

The 65 findings in the latest release are what GoldCheck would hand to the next reviewer. For 31 of them, 21 ties and 10 empty answers, the test is defective whatever the question intends. A tie leaves the answer to the engine, and an empty answer accepts any query that matches nothing. For the 32 fan-outs and the 5 NULL tops, the intent decides. Two authors judged these 37 findings independently from the question and its evidence note, without seeing each other's labels. They agreed on 33 of

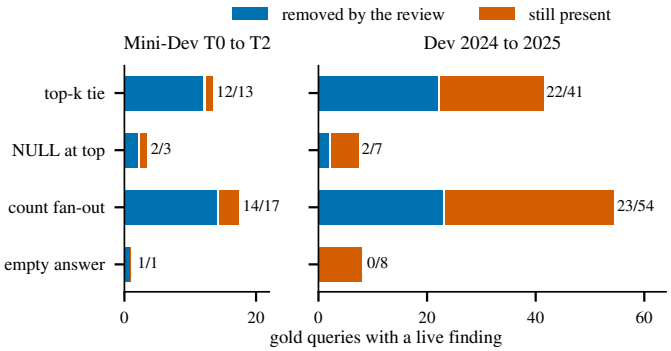


Fig. 6. Live findings per class in the version under review, split by whether the next version still has them.

TABLE IV
GoldCheck findings per release. A query can have several findings.

	Mini-Dev (498)			Dev (1,534)	
Finding	T0	T1	T2	2024	2025
Top-k tie	13	8	1	41	21
NULL at top	3	1	1	7	5
Count fan-out	17	5	3	54	32
Empty answer	1	0	0	8	10
Queries with a live finding	32	14	5	105	65
Queries with a latent witness	31	32	29	93	77
Top-k queries	75	74	68	250	240
Skipped (parser round trip)	26	25	23	56	51

the 37, with a Cohen's κ [32] of 0.60 and a 95% bootstrap interval from 0.16 to 0.92. The interval is wide because only 37 items were judged. The κ is lower than the raw agreement because most findings are errors, which raises chance agreement. Both judged all 5 NULL tops to be errors. Of the 32 fan-outs, both judged 24 to be errors and 4 to be acceptable. They disagreed on 4, and three of these are ratios whose other reading exceeds 100%. We leave these 4 unresolved. In total, 57 of the 65 flagged queries are a defective test or an error on which both annotators agree. Typical acceptable cases are counts meant to count join rows. In "How many posts did Jay Stevens have in 2010?", counting a user identifier over the posts table yields the number of posts.

E. Findings in the latest release

Table V shows typical findings in the 2025 release. By database, the share of questions with a finding ranges from 1.6% for the superhero database to 11.0% for the thrombosis database. Three patterns stand out.

First, one database dominates. The thrombosis database holds 18 of the 65 findings. Its patients have many lab tests each, and most of its findings are fan-outs over the join of patients and lab tests. Its questions ask about patients, but its gold queries often count lab rows. The database had 24 fan-outs in 2024 and still has 14.

Second, ties at the top are not rare corner cases. In 10 of the 21 tie findings, the gold keeps one row while two to four rows share the top value. In eight more, the tied group is larger, up to 561 rows. In the remaining three, the cut is at $k = 5$ or $k = 10$. A system that answers the question as asked lists all tied rows, and it receives no credit in any of these cases.

Third, empty answers cluster. Six of the ten ask about the 2008 or 2011 Australian Grand Prix in the motorsport database. Empty answers grew from eight to ten in the review, and the review changed the output of none of the eight. Editing the query cannot repair an empty answer. The question or the data must change.

Findings are not concentrated in hard questions. By BIRD's difficulty label, 3.6% of simple, 6.3% of moderate, and 2.6% of challenging questions have a live finding. A tie or a fan-out is as easy to write in a one-line query as in a nested one.

F. Targeted versus random state edits

Test suites search for distinguishing states by sampling [9]. GoldCheck attempted a witness on 116 Dev 2025 top-k queries. Its targeted search edits only rows just below the cut, and it found a witness for 77 with at most six one-cell edits. Random one-cell edits found 35 with one try, 52 within ten, and 66 within 100. They found nine witnesses that the targeted search missed and missed 20 that it found. A likely reason is that the edit that matters is predictable. It must promote a row that differs in output to the boundary, and a random row is often one that the query filters out. We did not test this explanation separately.

G. Contribution of each design choice

Table VI breaks the agreement down by class. Ties and fan-outs carry most of the signal in every review. The research audit changed six of the eight ties that the community review had left. The maintainers changed the output of 14 of 41 ties and 23 of 54 fan-outs, against 15% of queries without a finding.

Requiring a difference in output. In Dev 2024, 20 gold queries cut at a tie whose rows all print the same answer. An example is two lab records of the same patient with the same value. GoldCheck does not flag them. The review changed the output of one of them (5%), against 14 of 41 (34%) for ties whose rows differ. Flagging every tie would add 20 queries and lower the share of changed queries among tie flags from 34% to 25%.

Requiring execution. Static rules flag the same constructs without running them (Table III). They flag 4.7 to 11 times as many queries, and their lift falls to between 1.0 and 1.6. The construct alone is weak evidence. The data decide whether it matters.

H. Engine dependence of tie answers

Section ?? argued that a tie at the cut leaves the gold answer to the engine. We tested this directly. We ran

TABLE V
Typical GoldCheck findings in the 2025-11-06 BIRD Dev release, with abridged questions and the results of the gold query and of its other reading. A dash means that the class has no other reading.

ID	Question	Class	Gold	Other reading	Comment
802	Who is the tallest superhero?	tie	Surtur	Surtur, Ymir	two heroes share the height
766	Full name of the hero with the highest strength	tie	1 name	36 names	36 heroes share the top value
1002	Youngest racer and the race she first joined	tie	1 race	13 races	ordered by birth date only, not by race date
1196	Most common sign of patients with SLE	tie	1 sign	4 signs	four signs share the top count
37	Address of the school with the lowest excellence rate	NULL top	a school with no rate	561 such schools	the "lowest" rate is missing
1203	Female patients with white blood cells below 3.5	fan-out	641	88	counts lab tests, not patients
1308	Patients with a normal platelet level and other symptoms	fan-out	10,079	294	counts lab tests, not patients
150	Accounts in North Bohemia with a transaction to bank AB	fan-out	2,260	55	counts transactions, not accounts
677	How many posts did Jay Stevens have in 2010?	fan-out	3	1	acceptable, posts are meant
936	Fastest lap of Lewis Hamilton in the 2008 Australian GP	empty	no rows	–	any prediction with no rows scores
1504	Average total price of transactions in January 2012	empty	NULL	–	any average over no rows scores

TABLE VI
Changed in output by the next review, of the queries flagged per class. Entries are changed/flagged, and a dash means not measured. *Ties whose rows print the same answer, which GoldCheck does not flag.

Class	Community (T0)	Audit (T1)	Maintainers (Dev 2024)
Top-k tie	5/13	6/8	14/41
NULL at top	2/3	0/1	2/7
Count fan-out	12/17	2/5	23/54
Empty answer	1/1	–	0/8
Latent witness	2/31	4/32	12/93
Tie, same output*	–	–	1/20

TABLE VII
Gold queries whose result changed on identical data. DuckDB failed on 4 of 21 and 8 of 41 tie queries and is omitted for controls (dash). Numbers in parentheses are group sizes.

	Reverse scan	Rebuilt, reverse	Rebuilt, same order	DuckDB
Tie, Dev 2025 (21)	0	11	3	4
Tie, Dev 2024 (41)	0	23	6	7
No tie at cut, 2025 (206)	1	2	0	–
No tie at cut, 2024 (198)	1	3	0	–

every gold query with a live tie, unchanged, under four conditions that keep the data identical (Table VII). The first reverses the scan order of SQLite with PRAGMA reverse_unordered_selects. The second rebuilds every table the query reads in reverse row order, without indexes. The third rebuilds the tables in the original row order, which isolates the effect of the indexes. The fourth runs DuckDB 1.1.3 [33] on the same SQLite file.

In the 2025 release, rebuilding the tables in reverse order changed the gold answer of 11 of the 21 tie queries, and every new answer is another tied row. Dropping the indexes alone changed three. DuckDB returned another tied row for four of the 17 tie queries it could run. Over the three SQLite conditions, 12 of the 21 gold answers changed. In the 2024 release, 24 of 41 changed. Reversing the scan order changed none of them. Whether a tie flips thus depends on storage and plan details that the gold query does not control.

Among the 206 top-k gold queries without a tie at the cut, the SQLite conditions changed two. Both are state-dependent in another way. They select a column that is neither grouped nor aggregated, and SQLite takes its value from an arbitrary row. This construct is outside the four classes of GoldCheck.

I. LLM reviewers of growing size

If state-dependent gold queries were only a matter of reviewer skill, a larger model would remove them. We test this with SAR-Agent. Its released code calls OpenAI o3. To avoid API cost, we re-ran its released prompt and loop with open models from 3B to 120B parameters, Qwen2.5 [34], [35] and gpt-oss [36], served by vLLM [37] and Ollama [38]. The result describes these open models and does not transfer to o3, which we did not run. Three changes were unavoidable. The authors did not release

TABLE VIII
SAR-Agent on the 105 Dev 2024 gold queries with a live finding. "+ev." adds the GoldCheck evidence. "Named" means judged wrong with the mechanism named, by keyword match. Numbers in parentheses are class sizes, and a dash means not run.

	Qwen2.5			gpt-oss	32B	120B
Named, by class	3B	7B	32B	120B	+ev.	+ev.
Count fan-out (54)	6	3	19	43	49	46
Top-k tie (41)	1	3	10	11	27	37
Empty answer (8)	3	6	4	3	5	5
NULL at top (7)	0	0	0	2	6	5
Judged wrong (of 105)	39	28	59	79	100	95
Mean steps	3.9	3.3	4.1	1.3	–	–
Same verdict (of 50)	33	–	36	–	–	–
Mean latency (s)	17.0	6.0	81.5	18.7	–	–

TABLE IX
Predictions of four open models on the 2025 Dev release, scored by BIRD's rule and by a rule that also accepts the other reading of the 65 flagged gold queries.

	Qwen2.5			gpt-oss
	3B	7B	32B	120B
Accuracy, BIRD rule (%)	25.3	40.8	57.4	52.6
Accuracy, either reading (%)	25.4	41.1	58.4	53.5
On the 65 flagged questions				
scored correct, BIRD rule	9	14	20	16
of them empty answers	3	7	9	9
wrote the other reading	2	5	16	13
COUNT(DISTINCT) (of 32)	1	2	13	11
another or every tied row (of 21)	1	3	3	2

the per-database schema file, so we rebuild it from the CREATE statements and BIRD's column descriptions. Not every open model supports OpenAI function calling, so every model emits each call as one JSON object per reply. The released code opens the database writable and executes model-written SQL, so we run queries on a read-only connection. We review 300 Dev 2024 gold queries, the 105 with a live GoldCheck finding and 195 random others. For each finding, we ask whether SAR-Agent judged the gold wrong and named the mechanism in its explanation.

Fan-outs and ties respond differently to model size. Table VIII shows the result. As the model grows, SAR-Agent names more fan-outs. It names 6 of 54 with the 3B model and 43 of 54 with the 120B model. A reader can see a fan-out in the SQL text as a COUNT over a join without DISTINCT. Ties do not follow. From 32B to 120B, the count of named ties barely moves, from 10 to 11 of 41. Neither the query nor its result shows a tie. The reviewer sees one row and has to suspect that the cut dropped a second row with the same value. Then the reviewer has to write a query to check. The 120B model took 1.3 steps per review on average, including the final verdict. It rarely ran a verification query and judged mostly by reading. The smaller models ran queries but did not look for ties. For "Who is the tallest superhero?" the 3B model ran the gold, saw one name, and wrote that "the result [('Surtur',)] confirms that Surtur is the tallest superhero according to the given data." For the 641 patients it wrote that the count "matches the expected count based on the conditions provided."

Latent findings are out of reach for a reviewer that only queries the official database. On the official database, the gold and its other reading agree, so no query a reviewer runs there can show a difference. The difference exists only on a database that the reviewer never sees.

Verdicts vary between runs. On 50 queries reviewed twice with the same model and settings, the verdicts agreed on 33 (Cohen's $\kappa = 0.30$) with the 3B model and on 36 ($\kappa = 0.47$) with the 32B model. GoldCheck gives the same output on every run.

GoldCheck as input to a reviewer. Because the two do different jobs, the natural use is to run GoldCheck first and hand its evidence to whoever judges the question, a person or an LLM. We re-ran SAR-Agent on the 105 findings with the GoldCheck evidence appended to its prompt (Table VIII, right). The evidence lists the tied rows that the gold cuts off, or the two counts, as results of running SQL. A note says that the evidence does not tell which reading the question intends. With the evidence, the 32B model named the tie in 27 of 41 findings instead of 10, and the 120B model in 37 instead of 11. The 32B model with evidence named 87 of the 110 mechanisms, more than the 120B model without it (59). In these runs, adding the evidence raised the number of named ties more than a fourfold increase in model size did.

The evidence does not settle the reading. With it, both models judged almost every flagged gold query wrong, 100 and 95 of 105, whereas the maintainers changed the output of about a third. The 120B model kept 10 gold queries as correct, and the review changed none of them. Ten cases are too few to read much into. Whether a finding is an error depends on what the question intends. For this reason, GoldCheck reports evidence and leaves the decision to a person.

J. Case study: scores of model predictions

A finding matters downstream only if systems actually write the other reading. We asked four open models of growing size to write one SQLite query for each of the 1,534 questions of the 2025 release. The prompt is zero-shot and contains the schema with BIRD's column descriptions and the evidence note, and the temperature is 0. We scored the predictions in two ways (Table IX). The first is BIRD's rule. The second also accepts the other reading of a flagged gold query.

The two larger models wrote the other reading more often than the two smaller ones. On the 32 fan-out questions, the 32B model counted distinct entities 13 times and counted join rows, as the gold does, 6 times. The 120B model did so 11 and 4 times. BIRD's rule rejects every one of those 24 entity counts. Across the 65 flagged questions,

TABLE X
The 134 research-audit corrections of Mini-Dev, coded by the audit's four error classes [6] and by mechanism, top eight of 42. Bold mechanisms are in the scope of GoldCheck.

Error class	n (%)
Misunderstanding of data or schema	92 (69)
Domain-knowledge error	23 (17)
Question and SQL logic do not match	17 (13)
Output format	2 (1)
Mechanism	n
Age computed by year subtraction only	15
Missing filter for rows of the wrong kind or NULL	14
Top-row selection ignores ties	10
Missing DISTINCT after a fan-out join	9
Wrong source table	9
Aggregation at the wrong grain	5
Placeholder value treated as data (0 or '-')	5
Domain rule not applied	5

the number of predictions that match the other reading grows from 2 for the 3B model to 16 and 13 for the two strongest. For those two, it exceeds the number of flagged non-empty questions they get right under BIRD's rule, 11 and 7. Empty answers work the other way. Both strong models score 9 of the 10, although the database holds no answer to any of them.

Over the whole release, the effect is about one point of accuracy, because the findings cover 4% of the questions. It does not reorder these four models. The second rule is an upper bound. It also accepts the other reading where the gold reading is the intended one, as for the posts of Jay Stevens. This case study covers four zero-shot models and no published system, so it shows that the other reading occurs in real predictions and does not establish a trend with model quality.

K. Coverage and cost

GoldCheck covers a narrow slice by design. Its live findings flag 11% to 20% of the output changes made by each review (Table III). This matches the share of state-dependent mechanisms among the 134 research-audit corrections that we coded (Table X). Ties and missing DISTINCT after a fan-out account for 19 of them (14%). Most corrections fix a wrong table, a wrong filter, or a misread domain rule, and no data-only check can find these. GoldCheck should therefore run before a broad review, not instead of it.

On the latest Dev release, GoldCheck needs a median of 17 ms per query. The 95th percentile is 1.4 s, and the slowest query takes 37 s because the gold query itself is slow. The release takes 637 CPU seconds, about one minute on ten cores. GoldCheck does not attempt a witness for 87 top-k queries whose sort key is an expression or a column of a common table expression.

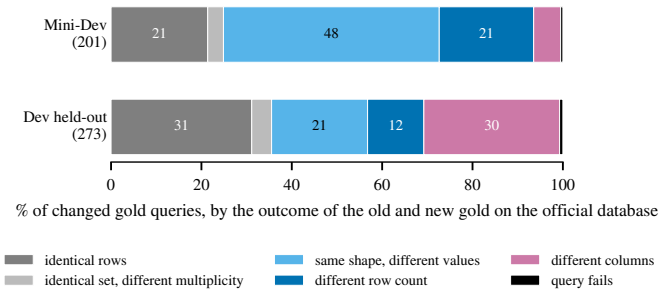


Fig. 7. Outcome of running each changed gold query and its replacement on the official database. Grey segments mark changes with an unchanged result set.

L. Case study: the training split

Gold queries also serve as training data for fine-tuned systems. We ran GoldCheck unchanged on the BIRD training split. The split has 9,428 gold queries over 69 databases, and we had looked at none of them before. The run took 16 minutes on 12 cores, with a median of 11 ms per query.

Of the 9,428 gold queries, 780 (8.3%) have a live finding. They comprise 337 empty answers, 260 ties at the cut, 183 count fan-outs, and 16 NULL tops. Another 476 have a latent witness. Findings occur in 67 of the 69 databases. The rate is higher than in Dev before its review (6.8%) and about twice the rate in Dev after it (4.2%). The training split has had no comparable review. The run also exposed a problem of another kind. In total, 367 training gold queries do not run at all, and 330 of them come from one database, retail_world. Its gold queries name tables and columns that its database does not contain, in 330 of its 373 queries. We did not measure the effect of these pairs on a fine-tuned system.

VI. Execution Results of Corrected Gold Queries

We ran every old and new gold pair on the official database (Fig. 7). On Mini-Dev, either review later changed 201 original gold queries. Of these, 93.5% run without error, return at least one row, and return as many columns as their replacement. Under BIRD's own scoring rule, 50 of the 201 (24.9%) return the same result set as the corrected query. On held-out Dev, 97 of 273 (35.5%) do. In the research audit alone, 59 of its 134 SQL corrections leave the result set unchanged. Such corrections cannot change any score under the benchmark's metric. Readers of error rates such as 52.8% should keep this in mind, and that rate also counts ambiguous questions [6]. A reviewer who checks the old gold by running it sees the same answer as after the fix.

A pair with the same result may still be non-equivalent. We re-ran each such pair on perturbed copies of the tables it references. The perturbations duplicate a few rows, set referenced columns to NULL, copy extreme sort-key values into other rows, move dates to a birthday boundary, and

TABLE XI
Old and new gold pairs with the same result on the official database, re-run on perturbed copies of the referenced tables.

	Mini-Dev	Held-out Dev	Control
Pairs tested	45	95	99
Duplicate rows	4	4	0
NULL values	6	8	0
Ties at the sort key	2	10	0
Birthday boundary	0	1	0
Delete 20% of rows	10	8	0
Exposed by any	14 (31.1%)	21 (22.1%)	0

delete a random 20% of rows, each with three seeds. The perturbations could run on 45 of the 50 hidden Mini-Dev changes and on 95 of the 97 held-out Dev changes. They separated 14 (31.1%) and 21 (22.1%) of them (Table XI). As a control, we paired 103 unchanged gold queries with their own parser-regenerated text. The perturbations ran on 99 of these pairs and separated none.

VII. Case Study: System Rankings

Jin et al. ran 16 systems on Dev-100 before and after correction and reported a Spearman correlation of 0.32 between the two rankings [6]. They describe the correction as the only factor that differs. For each system, their release contains one prediction file for the original inputs and one for the corrected inputs. Two facts about these files matter.

First, the authors regenerated the predictions instead of reusing them. The corrections changed the question in 19 examples and the evidence in 17, so at most 36 examples had new inputs [6]. Yet 12 of the 15 systems we checked differ between the two files on more than 36 examples, 49.5 on average and up to 85. Part of the reported ranking change therefore reflects run-to-run variation of LLM-based systems, not the corrections. Second, both released CHESS files are verbatim copies of the gold of the same version, for all 100 queries. The release thus cannot reproduce the reported change of CHESS from 62% to 81%, the largest in the study. We exclude CHESS.

To isolate the gold, we graded one fixed prediction file against both gold versions on the official databases (Table XII). Kendall τ [39] is 0.80 for the original-input predictions and 0.62 for the corrected-input predictions. Rankings do change, and one system moves 11 places, but less than the earlier analysis suggests. With 100 questions, the intervals are wide.

State-dependent gold queries add a specific bias. For a top-k tie, a system that returns every tied row never receives credit. A system that returns a single row receives credit only if the engine picks the same row as for the gold. For an empty answer, a system whose query matches nothing receives credit. In the 2025 Dev release, ties, NULL tops, and empty answers cover 33 questions (2.2%) before counting fan-outs.

TABLE XII
Fifteen systems on Dev-100, with one prediction file graded against the old and the corrected gold. τ is Kendall's rank correlation with a 95% bootstrap interval over questions.

Prediction file	original inputs	corrected inputs
Kendall τ	0.80 [0.60, 0.90]	0.62 [0.33, 0.79]
Mean score change (points)	9.9	5.5
Largest score change	13	10
Mean rank change	1.1	2.0
Largest rank change	4	11

VIII. Discussion and Lessons

Place in the pipeline. GoldCheck is cheap enough to run while an annotator writes a gold query, the way a linter runs on code. An annotator who sees "two rows are tied at the cut" or "this count is 641 rows but 88 patients" can fix the query, reword the question, or confirm the reading on the spot. The same check can run before every release and in continuous integration for a benchmark repository. Downstream, its findings help in two ways. Leaderboards can apply a tie-aware scoring rule. Builders of training sets that reuse gold queries can drop or repair flagged pairs. A model then does not learn to cut ties arbitrarily or to count join rows.

Combining with an LLM reviewer. GoldCheck and SAR-Agent do different jobs. GoldCheck finds a narrow class of problems with proof and no judgment. An LLM reviewer can judge whether a reading fits the question. In our runs, it rarely noticed that the data make two readings differ at a tie, and a larger model did not change this. A natural combination runs GoldCheck first and gives its evidence to the reviewer or to a person. The reviewer then no longer judges the whole query. It decides whether the question asks for the 641 rows or the 88 patients.

Ratios and averages. In 10 of the 32 fan-outs in the 2025 release, the count is part of a percentage or an average. There, the DISTINCT reading changes one side of the ratio. In six cases it yields a value above 100%. Such findings show that the count depends on the join, but the second reading is not a candidate answer. A finer rewrite would apply DISTINCT consistently to numerator and denominator.

More constructs. The four classes are a start. The engine experiment exposed a fifth class. A query selects a column that it neither groups nor aggregates, and SQLite takes the value from an arbitrary row. The second reading requires the grouping to determine the column functionally, and execution can test this in the same way. Integer division and case-sensitive string matching are further candidates whose meaning differs across engines.

Beyond BIRD and SQLite. The checks need only a parser, the gold query, and a database the checker can query. The tie and fan-out rewrites use window functions and COUNT(DISTINCT), which are standard SQL. Two parts of our implementation are specific to SQLite. It

addresses a row by its rowid, and it uses TEMP tables that shadow base tables of the same name. On other engines, a primary key and a session-local copy of the table serve the same purposes. Benchmarks that host their databases in a cloud warehouse, such as Spider 2.0-Snow [3], permit only the live checks unless a writable copy is available. The meaning of a NULL-top finding changes with the engine. A benchmark should therefore run this check on the engine it uses for scoring.

A. Lessons for benchmark builders

Run a state check before release, on the test and the training split, and hand its evidence to the reviewers.

Decide about ties. Either write the gold in a tie-complete form, or state that one row suffices and accept any tied row in scoring. The second option is a small change to the scorer. Let G be a gold query with a tie finding and G_c its tie-complete reading. The scorer accepts a prediction P if

$$P(D) = G_c(D), \text{ or}$$
$$|P(D)| = |G(D)| \text{ and } P(D) \subseteq G_c(D).$$

The first case accepts a system that lists every tied row. The second accepts any choice of k rows among them. The scorer already has the results that both cases need, and GoldCheck supplies G_c.

Do not ship questions with empty answers. Remove them, or mark them as unanswerable and score them accordingly.

Ship witnesses as extra database states. A latent witness is a one-cell change on which the gold and its other reading disagree. Test suites add such states for predictions [9]. Adding them for the gold makes the benchmark decide between readings instead of leaving the choice to the data.

Keep predictions fixed when measuring the effect of corrections. Re-running LLM-based systems mixes the effect of the gold with run-to-run variation. Release the exact predictions that the scorer graded, and check them.

IX. Conclusion and Future Work

A benchmark checks each gold query on one database state, and a gold query can return the expected answer there by accident. We defined state-dependent gold queries and built GoldCheck, a deterministic checker that proves each finding by execution. In three human reviews of the BIRD gold, the review changed the output of flagged queries 2.3 to 4.1 times as often as that of other queries. The latest official BIRD Dev release still contains 65 flagged queries. Tie answers changed when we rebuilt the tables, and LLM reviewers of up to 120B parameters named few ties without the evidence from GoldCheck.

Our results set the agenda for our next steps. (1) GoldCheck covers four classes and 11% to 20% of the output changes made by each review. We plan to add checks for ungrouped columns, integer division, and case-sensitive matching. (2) All releases come from BIRD and SQLite. We plan to port the checks to other engines and to benchmarks that publish per-example corrections. (3) Later releases are a reference, not ground truth, and two annotators judged 37 findings with a wide interval for κ. We plan a larger annotation with more annotators and a finer rewrite for ratio queries. (4) Our LLM comparison uses open models and keyword matching. We plan to run the original o3 configuration and to label its explanations by hand. (5) We measured the effect on scores with four zero-shot models on one release. An end-to-end study with published systems and a tie-aware scorer is the natural next experiment. (6) An LLM assistant produced the codes for the 134 audit corrections. We plan to re-code them by hand.

Data and Code Availability

All gold releases we use are public under CC BY-SA 4.0. Every artifact needed to reproduce the results is in a public repository at https://github.com/graphuofm/goldcheck. It holds the GoldCheck checker, the aligned tables of the five releases, the baselines, the SAR-Agent re-run and model predictions with all outputs and logs, the engine and training-split runs, the coding table, both annotators' labels, and the script behind each number, table, and figure. All models in our experiments have public weights, and no experiment needs a paid API.

AI-Generated Content Acknowledgement

We used an LLM assistant, Claude (Anthropic), to write code, run experiments, draft and edit text in all sections, and produce the codes in Table X. The authors reviewed the code, results, and text and take responsibility for the content. The 37 findings were labeled by two authors without AI assistance.

References

[1] B. Li, Y. Luo, C. Chai, G. Li, and N. Tang, "The dawn of natural language to SQL: Are we fully ready?" Proc. VLDB Endow., vol. 17, no. 11, pp. 3318–3331, 2024.

[2] A. Floratou et al., "NL2SQL is a solved problem… not!" in Proc. CIDR, 2024.

[3] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin, V. Zhong, C. Xiong, R. Sun, Q. Liu, S. Wang, and T. Yu, "Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows," in Proc. ICLR, 2025, arXiv:2411.07763.

[4] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. C. C. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs," in Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track, 2023.

[5] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in Proc. EMNLP, 2018, pp. 3911–3921.

[6] T. Jin, Y. Choi, Y. Zhu, and D. Kang, "Pervasive annotation errors break text-to-SQL benchmarks and leaderboards," Proc. VLDB Endow., 2026, arXiv:2601.08778.

[7] BIRD Team, "BIRD-SQL dev, update 2025-11-06," 2025, https://huggingface.co/datasets/birdsql/bird_sql_dev_20251106, CC BY-SA 4.0.

[8] B. Chandra, B. Chawda, B. Kar, K. V. M. Reddy, S. Shah, and S. Sudarshan, "Data generation for testing and grading SQL queries," The VLDB Journal, vol. 24, no. 6, pp. 731–755, 2015.

[9] R. Zhong, T. Yu, and D. Klein, "Semantic evaluation for text-to-SQL with distilled test suites," in Proc. EMNLP, 2020, pp. 396–411.

[10] M. Rigger and Z. Su, "Testing database engines via pivoted query synthesis," in Proc. USENIX OSDI, 2020.

[11] ——, "Finding bugs in database systems via query partitioning," Proc. ACM Program. Lang., vol. 4, no. OOPSLA, 2020.

[12] K. P. Gaffney, M. Prammer, L. Brasfield, D. R. Hipp, D. Kennedy, and J. M. Patel, "SQLite: Past, present, and future," Proc. VLDB Endow., vol. 15, no. 12, pp. 3535–3547, 2022.

[13] N. Wretblad, F. Riseby, R. Biswas, A. Ahmadi, and O. Holmström, "Understanding the effects of noise in text-to-SQL: An examination of the BIRD-Bench benchmark," in Proc. ACL (Short Papers), 2024, pp. 356–369.

[14] Arcwise, "Corrections to BIRD Mini-Dev (Arcwise-Plat release)," 2024, distributed with https://github.com/uiuc-kang-lab/text_to_sql_benchmarks.

[15] D. Chen, X. Wang, S. Ren, Q. Ma, P. Zhao, and A. Liu, "GBV-SQL: Guided generation and SQL2Text back-translation validation for multi-agent Text2SQL," in Proc. ACL (Long Papers), 2026, pp. 18 391–18 406.

[16] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive label errors in test sets destabilize machine learning benchmarks," in NeurIPS Datasets and Benchmarks Track, 2021, arXiv:2103.14749.

[17] H. Kim, B.-H. So, W.-S. Han, and H. Lee, "Natural language to SQL: Where are we today?" Proc. VLDB Endow., vol. 13, no. 10, pp. 1737–1750, 2020.

[18] G. Katsogiannis-Meimarakis and G. Koutrika, "A survey on deep learning approaches for text-to-SQL," The VLDB Journal, vol. 32, no. 4, pp. 905–936, 2023.

[19] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, "CodeS: Towards building open-source language models for text-to-SQL," Proc. ACM Manag. Data, vol. 2, no. 3, pp. 1–28, 2024.

[20] M. Pourreza and D. Rafiei, "Evaluating cross-domain text-to-SQL models and benchmarks," in Proc. EMNLP, 2023, pp. 1601–1611.

[21] H. Kim, T. Jeon, S. Choi, S. Choi, and H. Cho, "FLEX: Expert-level false-less EXecution metric for text-to-SQL benchmark," in Proc. NAACL, 2025, pp. 4448–4475.

[22] S. Chu, C. Wang, K. Weitz, and A. Cheung, "Cosette: An automated prover for SQL," in Proc. CIDR, 2017.

[23] Q. Zhou, J. Arulraj, S. Navathe, W. Harris, and D. Xu, "Automated verification of query equivalence using satisfiability modulo theories," Proc. VLDB Endow., vol. 12, no. 11, pp. 1276–1288, 2019.

[24] Z. Abedjan, X. Chu, D. Deng, R. C. Fernandez, I. F. Ilyas, M. Ouzzani, P. Papotti, M. Stonebraker, and N. Tang, "Detecting data errors: Where are we and what needs to be done?" Proc. VLDB Endow., vol. 9, no. 12, pp. 993–1004, 2016.

[25] X. Chu, I. F. Ilyas, S. Krishnan, and J. Wang, "Data cleaning: Overview and emerging challenges," in Proc. SIGMOD, 2016, pp. 2201–2206.

[26] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," Proc. VLDB Endow., vol. 10, no. 11, pp. 1190–1201, 2017.

[27] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "ActiveClean: Interactive data cleaning for statistical modeling," Proc. VLDB Endow., vol. 9, no. 12, pp. 948–959, 2016.

[28] T. Mao et al., "SQLGlot: Python SQL parser and transpiler," https://github.com/tobymao/sqlglot, accessed 2026-11-11.

[29] BIRD Team, "BIRD SQL Mini-Dev, 2026 update," 2026, https://huggingface.co/datasets/birdsql/bird_mini_dev, released 2026-10-28.

[30] B. Efron, "Bootstrap methods: Another look at the jackknife," The Annals of Statistics, vol. 7, no. 1, pp. 1–26, 1979.

[31] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.

[32] J. Cohen, "A coefficient of agreement for nominal scales," Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960.

[33] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in Proc. SIGMOD, 2019, pp. 1981–1984.

[34] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[35] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu et al., "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.

[36] OpenAI, "gpt-oss-120b and gpt-oss-20b model card," arXiv:2508.10925, 2025.

[37] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in Proc. SOSP, 2023, pp. 611–626.

[38] "Ollama," https://github.com/ollama/ollama, accessed 2026-11-11.

[39] M. G. Kendall, "A new measure of rank correlation," Biometrika, vol. 30, no. 1–2, pp. 81–93, 1938.